

CS6180 Homework 1

Code: <https://github.com/XinyiLiu777/CS6180/tree/main/hw1>

Code: q2_rope_plot.py, plot_losses.py, make_pdf.py, and modified nanoGPT/ (model.py, train.py, config/hw1_base.py, run_hw1.sh).

Q1. Convolution

1.1 Output size = $(4 - 3)/1 + 1 = 2 \times 2$.

1.2 `im2col` turns each 3×3 patch into a column, so X_col is 9×4 . Column (r,c) is $\text{vec}(X[r:r+3, c:c+3])$. With $w = \text{vec}(W)^T$ (1×9): $Y = w \cdot X_col$ (1×4), reshaped to 2×2 .

GPU advantage: the convolution becomes one big dense GEMM. That means highly optimized BLAS kernels (cuBLAS, Tensor Cores), regular memory access and high parallelism, and many filters and images handled in a single call. Cost: extra memory, because pixels are duplicated (36 vs 16 entries).

1.3 With $g = \text{vec}(G)$:

$\partial L / \partial X_col = w^T g$ (9×4), $\partial L / \partial X = \text{col2im}(w^T g)$, where `col2im` sums overlapping entries.

Equivalently, this is a full convolution of the zero-padded G with the 180° -rotated kernel:

$\partial L / \partial X = \text{conv}(\text{pad}(G, 2), \text{rot180}(W))$, i.e. $\partial L / \partial X[p,q] = \sum_{\{r,c\}} G[r,c] \cdot W[p-r, q-c]$.

Q2. RoPE

2.1 $(R_{-i} q)^T (R_{-j} k) = q^T R_{\{j-i\}} k$. For each of the 64 rotated pairs, with $a = 1/\sqrt{128}$:

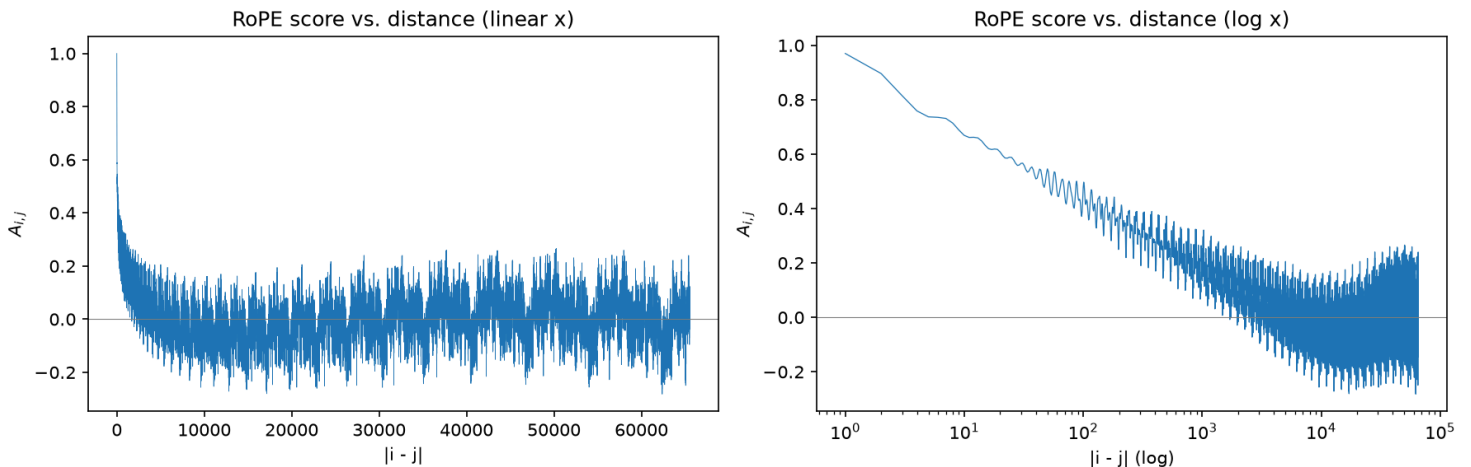
$[a, a] \cdot \text{Rot}(\Delta\theta_m) \cdot [a, a]^T = 2a^2 \cos(\Delta\theta_m)$ (the sin terms cancel).

Summing over the pairs:

$A_{\{i,j\}} = (1/64) \sum_{\{m=0\}^{63}} \cos((i-j) \theta_m)$, $\theta_m = 10000^{(-2m/128)}$

This is the score before the $1/\sqrt{d}$ scaling and softmax.

2.2



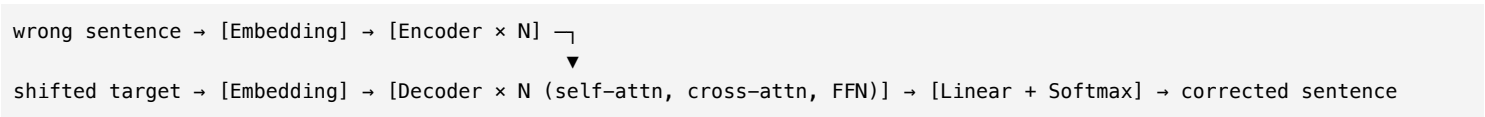
2.3 The score decays quickly with distance. Beyond a few thousand tokens it only oscillates around 0, so far positions carry no useful signal. Also, positions longer than the training length produce rotation angles the model never saw, so RoPE does not extrapolate past its

training context.

Mitigations: - Position Interpolation (rescale positions) - NTK-aware scaling / a larger RoPE base - YaRN - Fine-tuning on longer sequences - ALiBi as an alternative

Q3. Grammar error correction model

Architecture. A Transformer encoder-decoder (like BART/T5): - **Tokenizer:** BPE, 32k vocab. - **Embedding:** token embedding ($d = 768$) plus positional encoding. - **Encoder (6–12 layers):** bidirectional multi-head self-attention, then an FFN ($768 \rightarrow 3072 \rightarrow 768$, GELU), each with LayerNorm and a residual. It reads the whole erroneous sentence. - **Decoder (6–12 layers):** causal self-attention, cross-attention to the encoder output, and an FFN, each with LayerNorm and a residual. - **Output:** linear layer + softmax over the vocabulary.



Input / output. Input: the ungrammatical sentence. Output: the corrected sentence, generated token by token (a copy if it is already correct).

Training: - **Data:** start from a pretrained BART/T5, train on synthetic error pairs, then fine-tune on real GEC data (Lang-8, FCE, W&I+LOCNESS). - **Loss:** token-level cross-entropy with teacher forcing (label smoothing 0.1). - **Optimizer:** AdamW ($\beta = 0.9/0.98$, weight decay 0.01), lr $3e-5$ for fine-tuning with 1k warmup steps then linear decay, batch ~ 64 sentences, $\sim 20k-50k$ steps, early stopping on the dev set. - **Inference / evaluation:** beam search at inference; F0.5 (ERRANT) for evaluation.

Q4. Mini-LLM (nanoGPT)

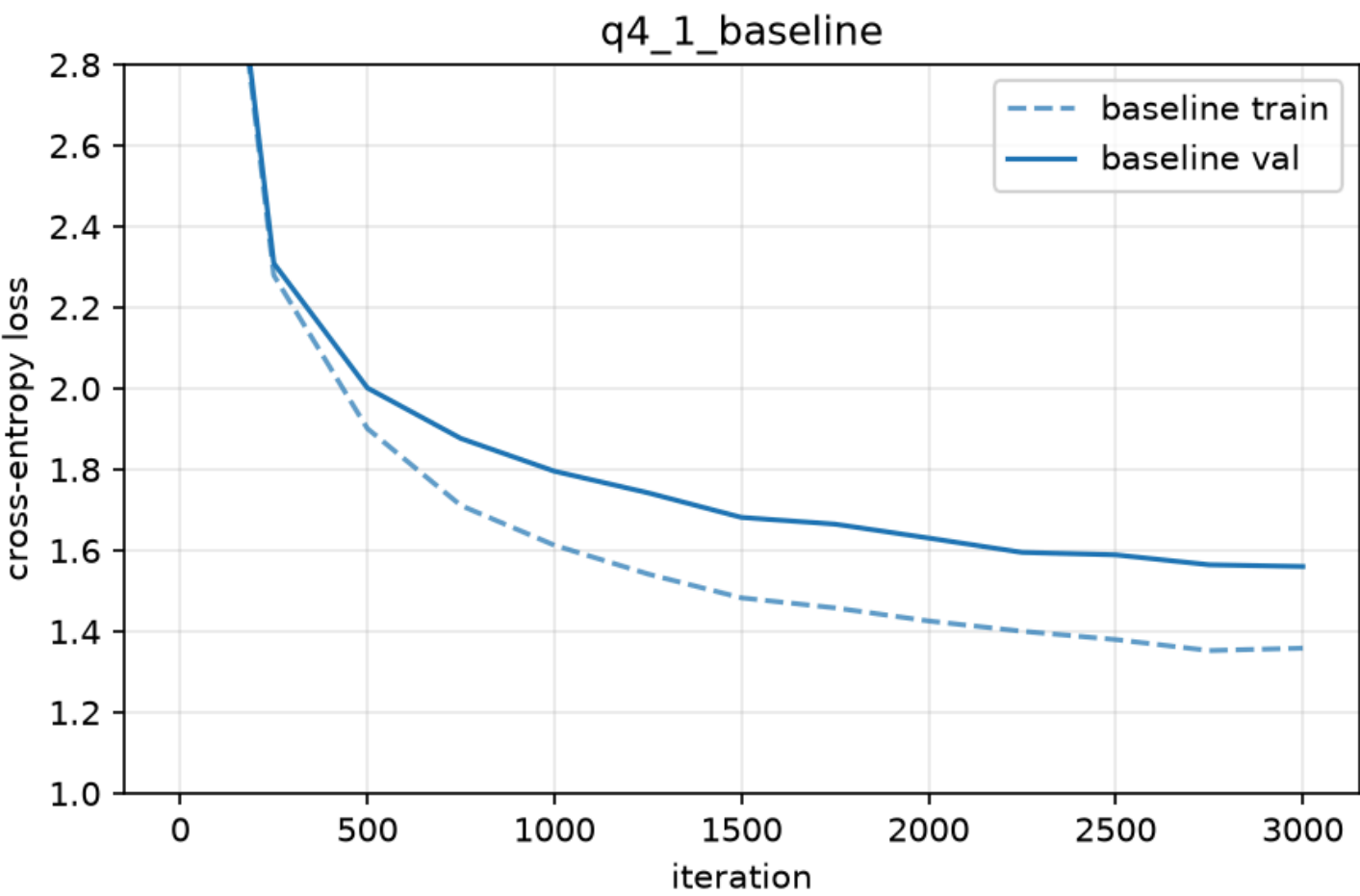
Setup. All runs share one config, nanoGPT/config/hw1_base.py: - Scaled-down train_shakespeare_char.py so all six runs fit on an 8GB Apple M1 (MPS): 4 layers, 4 heads, $d = 256$, context 128, batch 32, dropout 0.2. - 3000 iterations, lr $1e-3$ with cosine decay to $1e-4$. - Same seed (1337) for every run.

Each variant changes **one** flag in model.py relative to the baseline. The learning rate was **not tuned**: every run uses $1e-3$.

Run with `cd nanoGPT && python data/shakespeare_char/prepare.py && ./run_hw1.sh`, then `python plot_losses.py`. Logs are in logs/, loss history in nanoGPT/out-hw1/*/history.json.

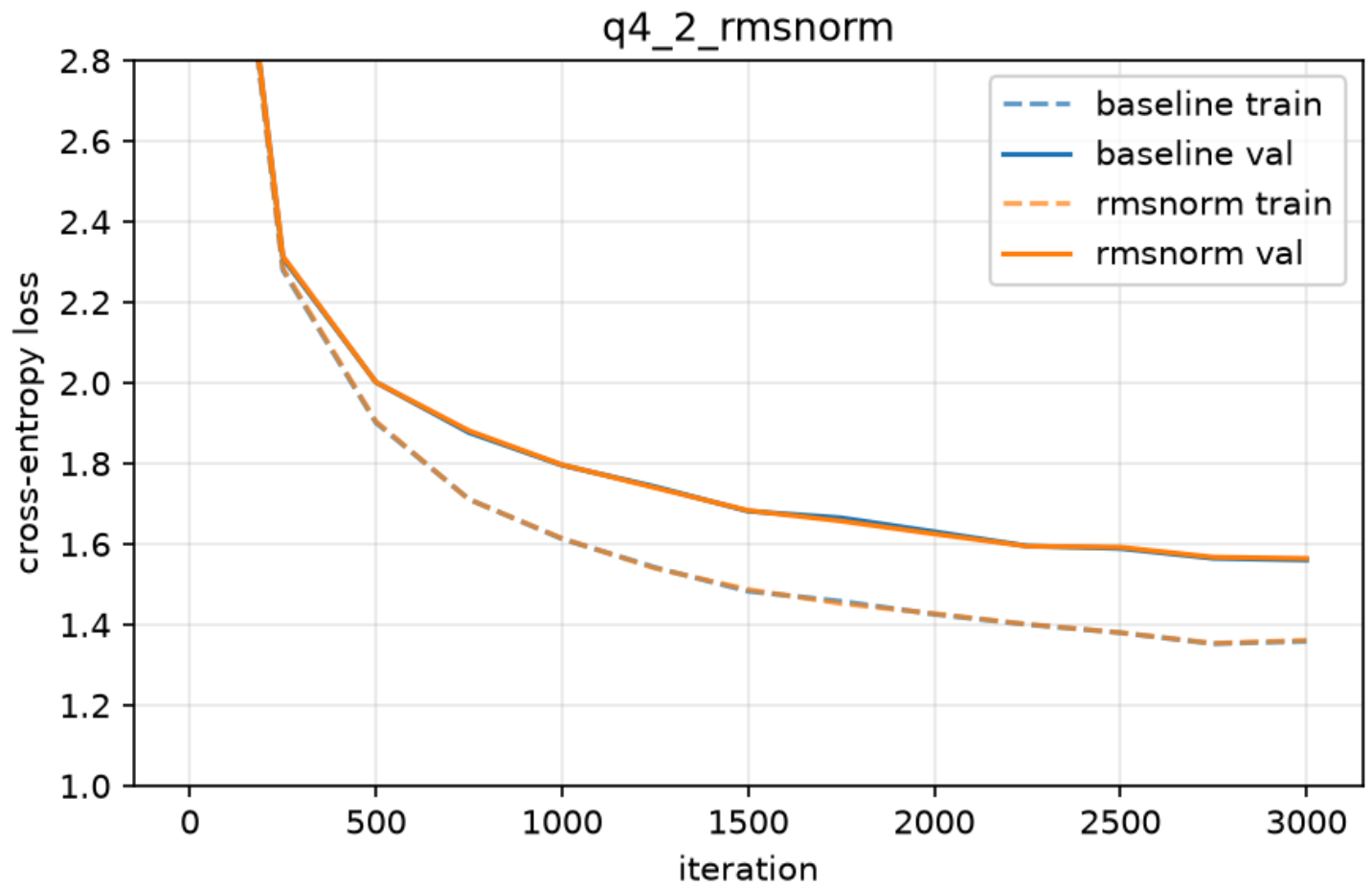
Run	Change	Params (M, excluding position embeddings)	Final train	Final val
baseline	—	3.16	1.3585	1.5602
rmsnorm	LayerNorm $\rightarrow$ RMSNorm	3.16	1.3612	1.5643
swiglu	GELU MLP $\rightarrow$ SwiGLU	3.23	1.3112	1.5353
nope	no positional encoding	3.16	1.6520	1.8393
rope	learned absolute $\rightarrow$ RoPE	3.16	1.2939	1.5170
gqa	MHA $\rightarrow$ GQA (4 query heads, 2 KV heads)	2.90	1.3679	1.5674

4.1 **Baseline.** Final val loss is 1.560.

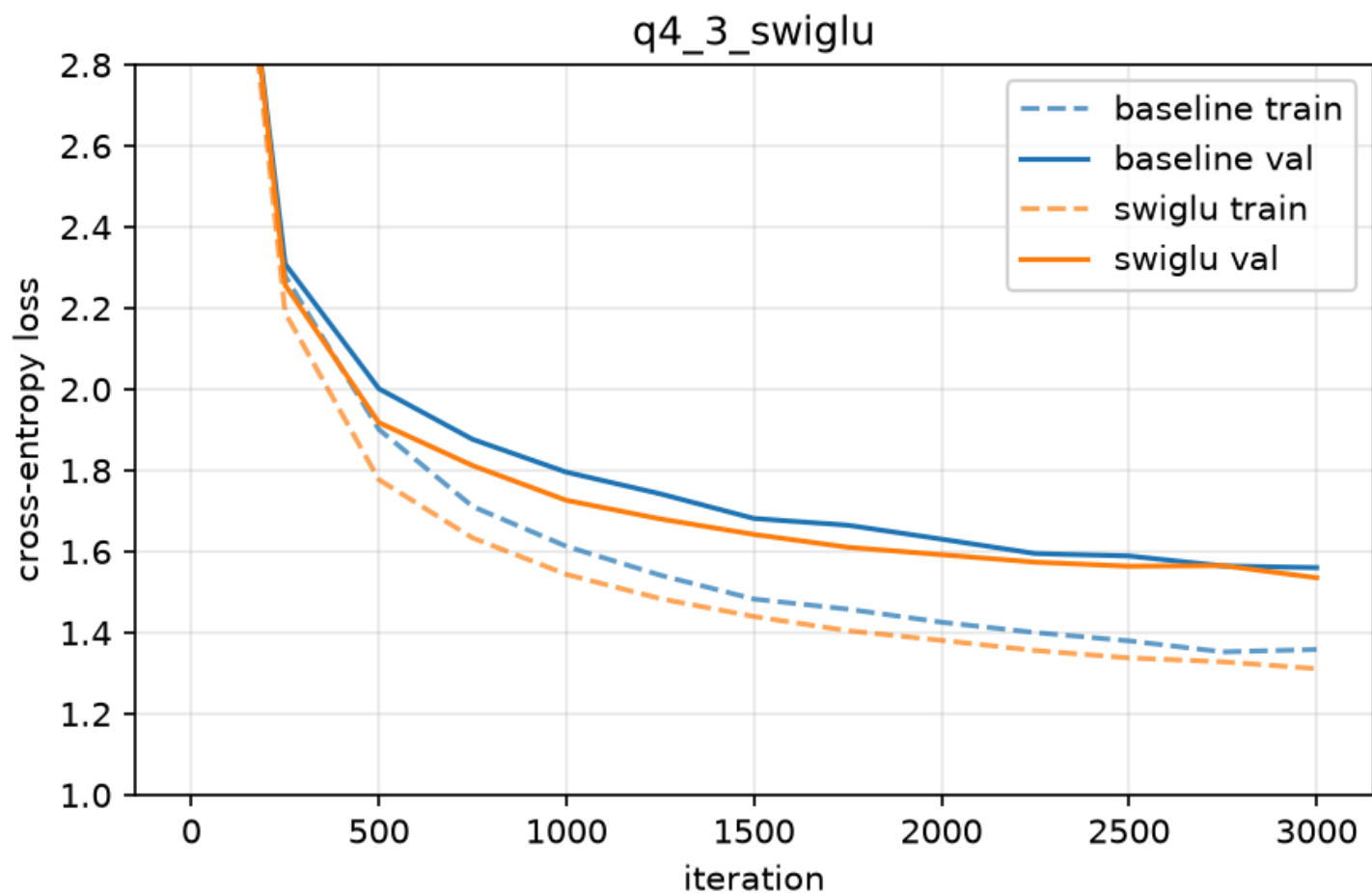


4.2 **RMSNorm.** nanoGPT uses **pre-LayerNorm**: $x = x + \text{attn}(\text{ln}_1(x))$, $x = x + \text{mlp}(\text{ln}_2(x))$, plus a final ln_f . Replacing every LayerNorm with RMSNorm (no mean subtraction, no bias) gives a val loss of 1.564 versus 1.560, essentially the same. RMSNorm is

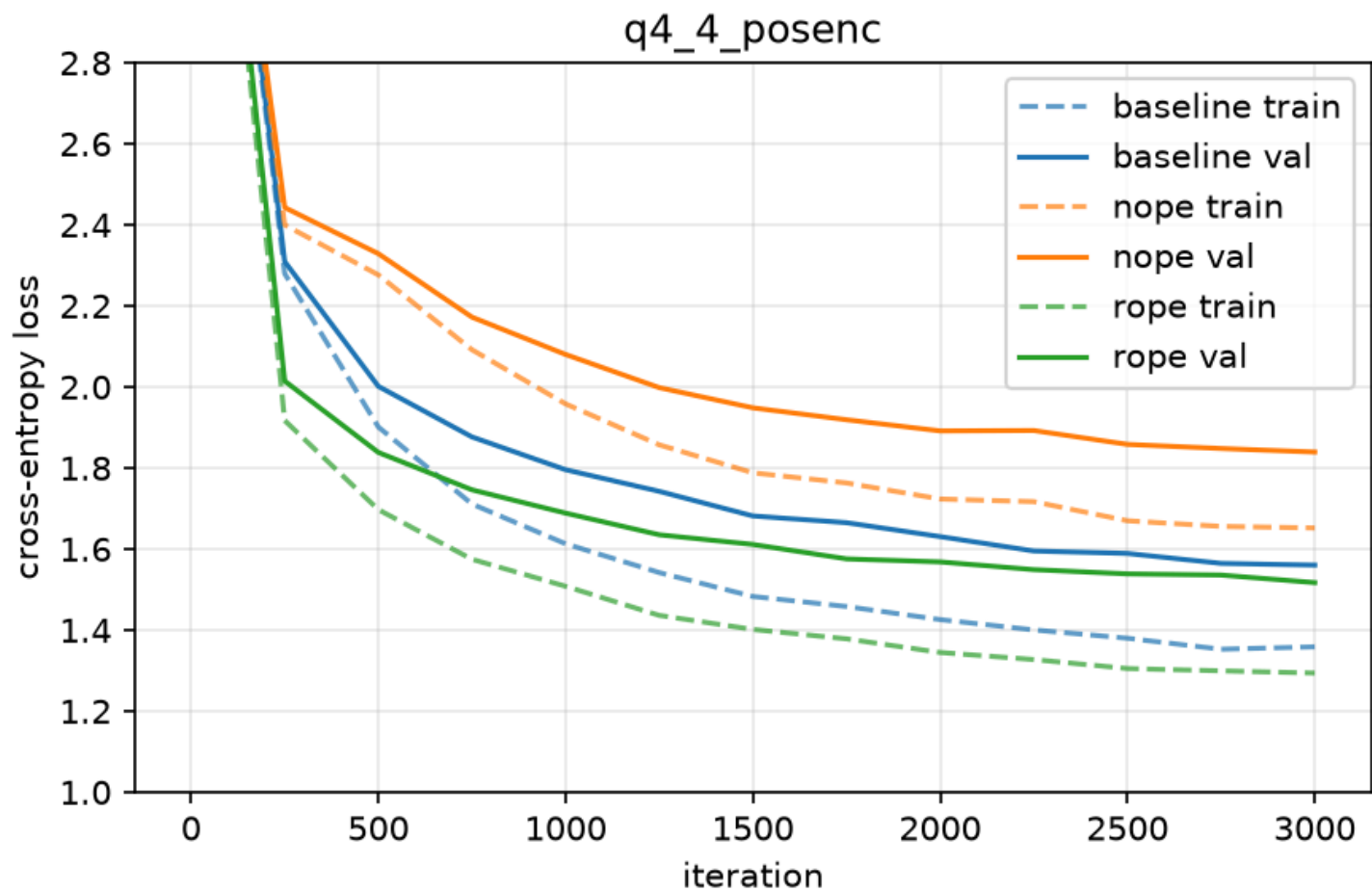
slightly cheaper with no loss in quality.



4.3 SwiGLU. The MLP uses **GELU** with a 4d hidden layer. SwiGLU computes $W_2(\text{SiLU}(W_1 \cdot x) \odot W_3 \cdot x)$ with hidden size $\approx 8/3 \cdot d$, rounded to 704, which gives roughly the same parameter count (+2%). Val loss improves to 1.535 (−0.025); the gating helps.



4.4 Positional encoding. The model uses **learned absolute positional embeddings** (wpe). - **NoPE** (remove wpe) is much worse: 1.839, +0.28. Causal masking gives only weak implicit position information, which is not enough for this small model. - **RoPE** (rotate q and k in every attention layer, base 10000) is the best run: 1.517, -0.043. Relative-position information injected at every layer helps character-level modeling.



4.5 GQA. Keys and values are projected to 2 heads, and each KV head is shared by 2 query heads (`repeat_interleave`). This cuts attention K/V parameters and the KV cache in half (~8% total parameters). Val loss is 1.567 versus 1.560: almost no quality loss for a

smaller model and cheaper inference.

